

JOVI — Plataforma de Produtividade Estudantil

Integrantes

- Gustavo Teotonio Silva
- Guilherme Men
- Matheus Tasso
- Alisson Gomes
- Vitor Kenji

Sumário

1. [Descritivo do Projeto](#)
 - 1.1 Objetivo Geral
 - 1.2 Justificativa
 - 1.3 Público-Alvo
 - 1.4 Arquitetura da Solução
 - 1.5 Stack Tecnológica
2. [Funcionalidades Implementadas](#)
 - 2.1 Análise de Imagem com IA
 - 2.2 Confirmação e Persistência de Conteúdo
 - 2.3 Tradução de Imagem
 - 2.4 Tradução de Texto
 - 2.5 Geração de Resumo por IA
 - 2.6 Dashboard de Conteúdos Recentes
 - 2.7 Gerenciamento de Pastas
 - 2.8 Gerenciamento de Matérias
 - 2.9 Cache Temporário de Imagens
 - 2.10 Soft Delete

1. Descritivo do Projeto

1.1 Objetivo Geral

O JOVI é uma API backend projetada para auxiliar estudantes na organização e digitalização de seus materiais de estudo. A plataforma permite que o aluno fotografe anotações de caderno, extraia o texto automaticamente via inteligência artificial, organize o conteúdo por matéria em pastas, e gere resumos estruturados — tudo de forma automatizada e inteligente.

O objetivo central é reduzir o tempo gasto com organização manual de conteúdo acadêmico, permitindo que o estudante foque no aprendizado enquanto a plataforma cuida da catalogação, extração e síntese das informações.

1.2 Justificativa

Estudantes universitários e do ensino médio frequentemente acumulam grande volume de anotações em cadernos físicos, fotos de quadros e materiais dispersos. A falta de organização digital desse conteúdo resulta em:

- Dificuldade para localizar informações antes de provas
- Perda de contexto entre aulas
- Tempo excessivo gasto reorganizando materiais
- Impossibilidade de buscar conteúdo textualmente em fotos

O JOVI resolve esses problemas ao oferecer um pipeline automatizado: o aluno tira uma foto, a IA extrai o texto, sugere a matéria correspondente, e o sistema organiza tudo em pastas categorizadas. Adicionalmente, a geração de resumos por IA permite revisões rápidas e objetivas.

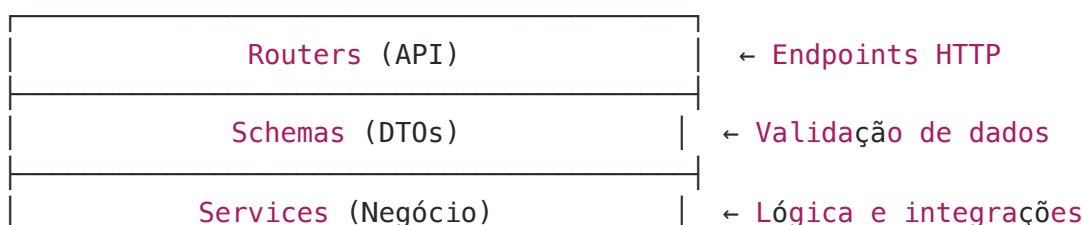
A solução não é genérica — cada funcionalidade foi desenhada especificamente para o fluxo de estudo: captura → extração → organização → revisão.

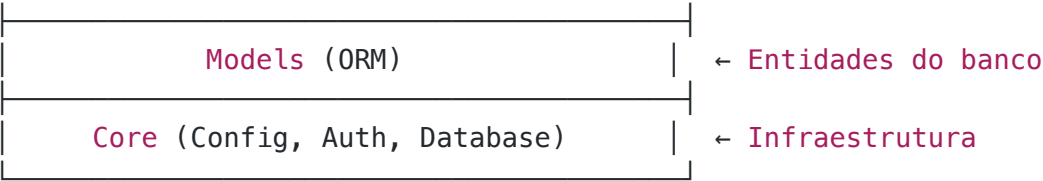
1.3 Público-Alvo

- Estudantes do ensino médio e universitários
- Alunos que utilizam cadernos físicos e desejam digitalizar suas anotações
- Estudantes que buscam ferramentas de revisão automatizada

1.4 Arquitetura da Solução

O projeto segue uma Layered Architecture (arquitetura em camadas) com separação clara de responsabilidades:





1.5 Stack Tecnológica

Componente	Tecnologia
Linguagem	Python 3.12 com Type Hints
Framework	FastAPI + Uvicorn
ORM	SQLAlchemy 2.0 (Declarativo)
Banco de Dados	PostgreSQL
Storage	Supabase Storage
IA (Visão)	Groq — Llama 4 Scout 17B
IA (Texto)	Groq — Llama 3.3 70B
Documentação	Swagger UI + Scalar
Deploy	Docker + Render

2. Funcionalidades Implementadas

2.1 Análise de Imagem com IA

Endpoint: POST /ia/analisar-imagem

O estudante envia uma foto de suas anotações (caderno, quadro, apostila). A IA processa a imagem utilizando o modelo de visão Llama 4 Scout e retorna:

- O texto completo extraído da imagem
- A matéria sugerida (ex: "Química", "História")
- Um `cache_id` para uso posterior na confirmação

Se a matéria sugerida pela IA não existir no banco, ela é criada automaticamente. A imagem fica armazenada em cache na memória do servidor por 5 minutos, evitando que o usuário precise reenviá-la na etapa de confirmação.

2.2 Confirmação e Persistência de Conteúdo

Endpoint: POST /conteudo/confirmar

Após revisar o resultado da análise, o estudante confirma o conteúdo enviando apenas o `cache_id`, o `id_materia` e o `texto_extraido`. O sistema então:

1. Recupera a imagem do cache (sem reenvio)
2. Faz upload para o Supabase Storage, organizando em pastas por matéria (ex: `quimica/uuid.jpg`)
3. Cria a pasta no banco de dados (se não existir para aquela matéria)
4. Persiste o conteúdo e vincula a imagem

Se o upload falhar, nenhuma alteração é feita no banco (garantia de consistência).

2.3 Tradução de Imagem

Endpoint: POST /ia/traduzir-imagem

Recebe uma imagem contendo texto em outro idioma, extrai o conteúdo visível e traduz para português brasileiro. Útil para estudantes que lidam com materiais em inglês, espanhol ou outros idiomas.

2.4 Tradução de Texto

Endpoint: POST /ia/traduzir-texto

Recebe uma string de texto e o idioma de destino (padrão: português brasileiro). Retorna a tradução gerada pelo modelo Llama 3.3 70B. O idioma de destino é configurável, permitindo traduções para qualquer língua.

2.5 Geração de Resumo por IA

Endpoint: POST /ia/resumo

Recebe o ID de um conteúdo já salvo no banco, recupera o texto original e gera um resumo estruturado em tópicos via IA. O resumo é salvo automaticamente no campo `resumo_ia` do conteúdo, ficando disponível para consultas futuras sem necessidade de reprocessamento.

2.6 Dashboard de Conteúdos Recentes

Endpoint: GET /dashboard/recentes

Retorna os 4 conteúdos mais recentes do usuário, incluindo:

- ID do conteúdo e da pasta
- Texto extraído e resumo (se existir)
- URL da primeira imagem vinculada

- Data da última atualização

Permite que o app mobile exiba rapidamente os últimos materiais estudados.

2.7 Gerenciamento de Pastas

Endpoints:

- GET /pastas — Lista pastas do usuário com contagem de conteúdos
- GET /pastas/{id} — Detalhe da pasta com todos os conteúdos e imagens
- DELETE /pastas/{id} — Soft delete da pasta e conteúdos filhos

As pastas são criadas automaticamente ao confirmar um conteúdo (uma pasta por matéria). A listagem inclui a quantidade de arquivos dentro de cada pasta via subquery otimizada.

2.8 Gerenciamento de Matérias

Endpoints:

- GET /materias — Lista todas as matérias cadastradas
- POST /materias — Cria nova matéria
- DELETE /materias/{id} — Remove matéria

O sistema vem com 26 matérias pré-cadastradas (seed automático no startup), cobrindo ensino médio e superior. Novas matérias são criadas automaticamente quando a IA sugere uma que não existe.

2.9 Cache Temporário de Imagens

Para otimizar o uso de banda (especialmente em dispositivos móveis), a imagem enviada na análise é armazenada em cache na memória do servidor com TTL de 5 minutos. Isso elimina a necessidade de reenvio na confirmação. Um job assíncrono limpa entradas expiradas a cada 60 segundos, garantindo que o servidor não acumule dados desnecessários.

2.10 Soft Delete

A exclusão de pastas e conteúdos utiliza soft delete — os registros são marcados com `deletado=True` e `deletado_em=timestamp`, mas permanecem no banco. Isso garante:

- Possibilidade de recuperação de dados
- Auditoria de ações do usuário
- Imagens permanecem no Supabase Storage (sem perda de arquivos)

Todas as queries de listagem filtram automaticamente registros deletados.

2.11 Interface de Linha de Comando (CLI)

Arquivo: `cli.py`

O projeto inclui um programa interativo de terminal que consome a API REST, permitindo ao usuário acessar todas as funcionalidades via menu de opções. O CLI implementa os seguintes conceitos de programação:

- **Menu de opções** com `loop while True`
- **Estrutura de decisão** `match-case` para direcionar funcionalidades
- **Entrada de dados** via `input()` com validação
- **Saída formatada** com `f-strings`
- **Manipulação de listas** para exibir matérias, pastas e conteúdos
- **Repetição** com `for` para iterar sobre resultados da API

Opções disponíveis no menu:

Opção	Funcionalidade
1	Analisar imagem (extrair texto e sugerir matéria)
2	Confirmar conteúdo (salvar no sistema)
3	Traduzir texto
4	Gerar resumo por IA
5	Listar matérias
6	Listar pastas do usuário
7	Ver conteúdos recentes
0	Sair

Cada funcionalidade valida os dados de entrada, chama o endpoint correspondente da API, trata erros de conexão e exibe o resultado formatado ao usuário. Após a execução, o programa retorna automaticamente ao menu principal.

3. Como Executar o Projeto

Pré-requisitos

- Python 3.12+
- PostgreSQL (local via Docker ou remoto)
- Conta no Supabase (Storage)
- API Key do Groq (gratuita)

Passo a passo

```
# 1. Clone o repositório
git clone https://github.com/jovi-insight/backend.git
cd backend

# 2. Crie e ative o ambiente virtual
python3.12 -m venv venv
source venv/bin/activate

# 3. Instale as dependências
pip install -r requirements.txt

# 4. Configure as variáveis de ambiente
cp .env.example .env
# Edite o .env com suas credenciais

# 5. Inicie a API (terminal 1)
uvicorn app.main:app --reload

# 6. Execute o CLI (terminal 2)
python3 cli.py
```

Execução via Docker

```
docker build -t jovi-app .
docker run -p 8000:8000 --env-file .env jovi-app
```

Documentação interativa

- Swagger UI: <http://localhost:8000/docs>
- Scalar: <http://localhost:8000/scalar>